

Contents

1	01-Introduction.md	4
2	Introduction	4
2.1	Before You Begin	4
2.2	What This Guide Covers	5
2.3	Optimization Goals	5
2.4	Recommended Workflow	5
2.5	Documentation Philosophy	5
2.6	Credits	6
2.6.1	Original Guide	6
2.6.2	Community Edition	6
3	02-Canta-and-Shizuku.md	6
4	Debloating with Canta and Shizuku	6
4.1	Prerequisites	6
4.2	Start the Shizuku Service	7
4.3	Grant Permission to Canta	7
4.4	Selecting Packages	9
4.5	Package Categories	9
4.5.1	System Applications	9
4.5.2	Google Components	9
4.5.3	Android Components	9
4.5.4	Android Themes	9
4.6	Why Debloat?	11
4.7	Reboot the Device	11
4.8	Verification Checklist	12
5	03-System-Settings.md	12
6	Recommended System Settings	12
6.1	Google Account	12
6.1.1	Disable Contact Synchronization	12
6.2	Google Play	12
6.2.1	Disable Play Protect	12
6.2.2	Steps	12
6.3	Location Services	13
6.3.1	Disable Location	13
6.3.2	Disable Wi-Fi and Bluetooth Scanning	13
6.4	Account Synchronization	13
6.4.1	Disable Automatic Sync	13
6.5	System Logging	13
6.5.1	Disable Logging	13
6.6	Background Applications	14
6.6.1	Restrict Background Battery Usage	14
6.7	Developer Options	14
6.7.1	Disable Window Animations	14
6.7.2	Keep Hardware Overlays Enabled	14
6.8	Before Continuing	14
7	04-Advanced-ADB-Toolkit.md	15
8	Advanced ADB Optimization Toolkit	15
9	Google Play Services	15
9.1	Restrict Google Play Services	15
9.1.1	Commands	15
9.2	Restrict Google Play Store	15

9.2.1	Commands	15
9.3	Restrict Gboard	16
9.3.1	Commands	16
9.4	Restrict OTA Updater	16
9.4.1	Commands	16
9.5	Frontend Protection	16
9.5.1	Daijisho (GitHub)	16
9.5.2	Daijisho (Google Play)	16
10	Performance Optimizations	16
10.1	Force Ahead-of-Time Compilation	16
10.1.1	Command	17
10.2	Compile a Single Application	17
10.2.1	Command	17
10.3	Execute Background Package Optimization	17
10.3.1	Command	17
10.4	Disable Application Error Reports	17
10.4.1	Command	17
10.5	Disable Security Reports	17
10.5.1	Command	17
10.6	Disable Background Sync During Battery Saver	17
10.6.1	Command	17
11	System Resource Management	18
11.1	Reduce Background Job Scheduling	18
11.1.1	Command	18
11.2	Enable Cached App Freezer	18
11.2.1	Command	18
11.3	Prioritize System UI	18
11.3.1	Command	18
11.4	Disable Captive Portal Detection	18
11.4.1	Command	18
11.5	Trim Package Cache	19
11.5.1	Command	19
11.6	Reduce Alarm Wakeups	19
11.6.1	Command	19
11.7	Disable Activity Recognition	19
11.7.1	Command	19
11.8	Disable Background Usage Statistics	19
11.8.1	Command	19
11.9	Disable Device Identifier Access	19
11.9.1	Command	19
11.10	Background Resource Management	19
11.11	Verification	20
12	Google Play Services Hardening	20
12.1	Disable Background Location Access	20
12.1.1	Commands	20
12.2	Disable Activity Recognition	20
12.2.1	Command	20
12.3	Disable Usage Statistics	20
12.3.1	Command	20
12.4	Disable Device Identifier Access	21
12.4.1	Command	21
12.5	Disable Wi-Fi and Bluetooth Background Scanning	21
12.5.1	Commands	21
12.6	Restrict Wake Locks	21
12.6.1	Command	21
12.7	Restrict Automatic Startup	21

12.7.1 Command	21
12.8 Restrict Overlay Windows	21
12.8.1 Command	21
12.9 Disable Google Feedback	21
12.9.1 Command	21
12.10 Disable Google Turbo	22
12.10.1 Command	22
12.11 Enable Sensor Privacy	22
12.11.1 Commands	22
13 Expected Result	22
13.1 Verification	22
14 Display & User Experience	22
14.1 Disable Window Animations	23
14.2 Keep Hardware Overlays Enabled	23
14.3 SurfaceFlinger Optimizations	23
14.4 Reboot the Device	23
15 Final Validation	23
16 Troubleshooting	24
16.1 Google Play Store Stops Updating	24
16.2 Unexpected Application Behavior	24
16.3 Increased Battery Consumption	24
17 Maintenance	24
17.1 After Updating an Emulator	24
17.2 After Major Firmware Updates	25
17.3 Periodic Review	25
18 Completion Checklist	25
19 05-Game-Driver.md	25
20 Game Driver Configuration	25
20.1 Open Game Driver Preferences	25
20.2 Configure Each Emulator	26
20.3 Validate the Configuration	26
20.4 Compatibility Notes	26
20.5 Final System Validation	26
21 Optimization Complete	27
22 Ongoing Maintenance	27
22.1 Completion Checklist	27
23 06-Appendices.md	27
24 Appendices	27
25 Appendix A — Common Package Names	28
26 Appendix B — Common ADB Commands	28
26.1 Compile All Installed Applications	28
26.2 Compile a Single Application	28
26.3 Execute Background Package Optimization	28
26.4 Trim Package Cache	28
26.5 Reboot the Device	28

27 Appendix C — Glossary	28
27.1 ADB	28
27.2 AOT	29
27.3 JIT	29
27.4 App Standby Bucket	29
27.5 AppOps	29
27.6 Background Restriction	29
27.7 Canta	29
27.8 Developer Options	29
27.9 Shizuku	29
27.10 SurfaceFlinger	29
27.11 Hardware Composer	29
27.12 Wake Lock	30
28 Troubleshooting	30
28.1 Google Play Store Does Not Update Applications	30
28.2 Google Play Services Displays Errors	30
28.3 Emulator Performance Did Not Improve	30
28.4 System Behaves Unexpectedly	30
28.5 Firmware Update Reset Some Settings	30
29 Credits	31
29.1 Original Guide	31
29.2 Community Edition	31
30 Final Notes	31

1 01-Introduction.md

2 Introduction

Welcome to the **PAM Stock OS Optimization Guide**.

This guide is based on **Uri's original optimization guide**, expanding upon Cyberyellowninja's *Dark Arts Guide*. The Community Edition reorganizes the content into a structured Markdown format, improving readability, navigation and long-term maintainability while preserving the original technical recommendations.

The purpose of this guide is to optimize the **PAM Stock OS** for handheld gaming by reducing unnecessary background activity, improving battery life and maximizing emulation performance without sacrificing the Stock OS experience.

2.1 Before You Begin

This guide assumes you are familiar with:

- Android Debug Bridge (ADB)
- Android Developer Options
- Shizuku
- Basic Android system configuration

If you are not comfortable using these tools, take some time to become familiar with them before continuing.

[!WARNING] The optimizations described in this guide modify Android system behavior.

Always review commands before executing them and make sure you understand their purpose.

You are responsible for any changes made to your device.

2.2 What This Guide Covers

The optimization process is divided into six sections:

1. **Debloating with Canta and Shizuku**
2. **Recommended System Settings**
3. **Advanced ADB Optimization Toolkit**
4. **Game Driver Configuration**
5. **Maintenance**
6. **Appendices**

Each section builds upon the previous one, and following the guide in order is strongly recommended.

2.3 Optimization Goals

The recommendations in this guide are intended to:

- Reduce unnecessary background activity.
- Improve battery life.
- Reduce background memory usage.
- Improve emulator responsiveness.
- Preserve Google Play compatibility.
- Maintain the Stock OS experience.

Results may vary depending on:

- Firmware version
 - Installed applications
 - Emulator versions
 - Individual usage patterns
-

2.4 Recommended Workflow

For the best results, complete the guide in the following order:

1. Prepare the device.
2. Remove unnecessary packages using Canta.
3. Configure the recommended Android settings.
4. Apply the Advanced ADB optimizations.
5. Configure the Game Driver.
6. Reboot the device.
7. Verify system functionality.
8. Test your emulators.

Skipping steps may reduce the overall effectiveness of the optimization process.

[!NOTE] Most optimizations only need to be applied once.

After major firmware updates or a factory reset, some settings may need to be configured again.

2.5 Documentation Philosophy

The Community Edition follows a simple editorial principle:

Preserve the knowledge. Improve the documentation.

The objective is to make the guide easier to read, review and maintain without changing the original optimization strategy.

Whenever editorial changes have been made, they are limited to:

- Formatting
- Organization
- Grammar
- OCR corrections
- Documentation structure

The technical recommendations remain those of the original author unless explicitly documented.

2.6 Credits

2.6.1 Original Guide

Uri

Responsible for:

- Research
- Testing
- Optimization methodology
- Technical documentation

2.6.2 Community Edition

BruhMeh

Responsible for:

- Editorial review
 - Markdown conversion
 - Documentation formatting
 - OCR corrections
 - Repository organization
 - Publishing support
-

Continue to [[02-Canta-and-Shizuku]]

3 02-Canta-and-Shizuku.md

4 Debloating with Canta and Shizuku

Removing unnecessary system applications is one of the most effective ways to reduce background activity, improve battery life and free system resources for emulation.

This guide uses **Shizuku** together with **Canta** to safely uninstall or disable packages without requiring root access.

[!IMPORTANT] The package selections in this guide are based on Uri's tested configuration.

Review every package before removing it and keep any application that you personally use.

4.1 Prerequisites

Before continuing, make sure the following requirements are met:

- Developer Options are enabled.

- USB Debugging is enabled.
 - Wireless Debugging is enabled.
 - Shizuku is installed.
 - Canta is installed.
-

4.2 Start the Shizuku Service

Launch **Shizuku**.

Verify that the service is running correctly before opening Canta.

If this is your first time using Shizuku:

1. Pair the device using Wireless Debugging.
2. Start the Shizuku service.
3. Confirm that the status is **Running**.

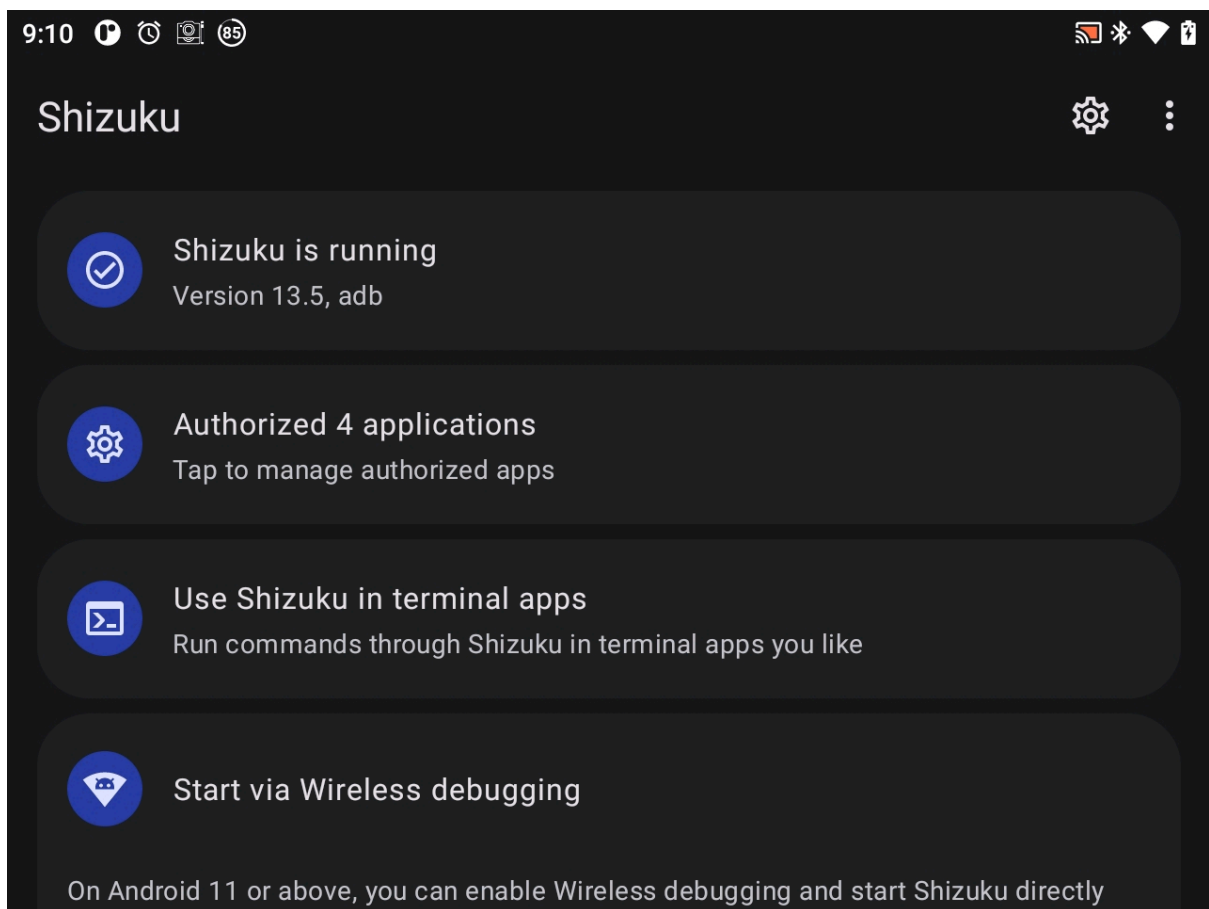


Figure 1: Shizuku running

Shizuku should be running before continuing.

4.3 Grant Permission to Canta

Open **Canta**.

When prompted, grant permission through Shizuku.

Once permission has been granted, Canta will display the list of installed packages.

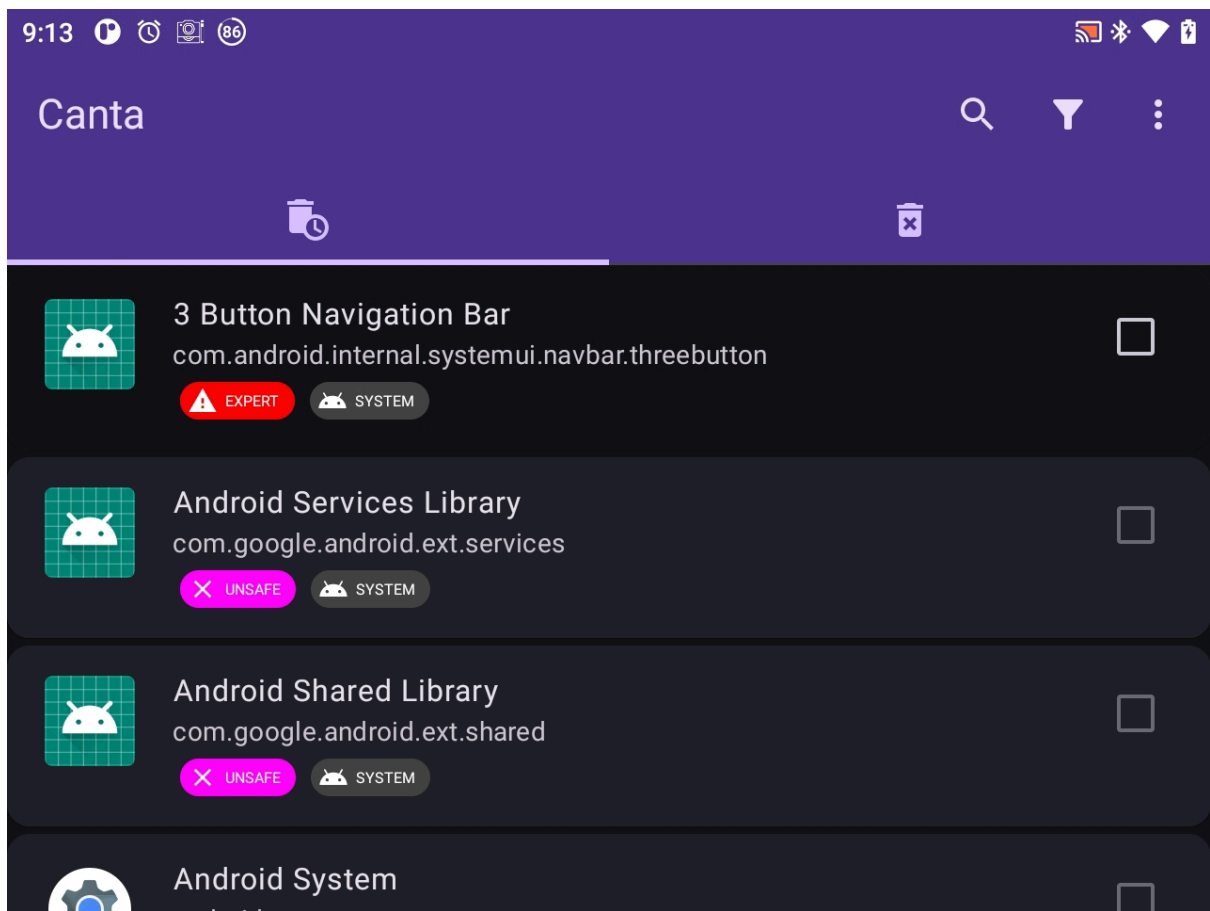


Figure 2: Granting permission

If permission was set correctly, this list of packages should show.

4.4 Selecting Packages

This screenshot shows all the recommended package selection.

Follow this list carefully.

Example of recommend package selection.

4.5 Package Categories

For readability, packages are grouped into logical categories.

4.5.1 System Applications

Applications that are not required for a typical handheld gaming experience.

Examples include:

```
com.android.music  
com.android.egg  
com.android.dreams.basic  
com.android.emergency
```

4.5.2 Google Components

Optional Google applications and services that are not required by every user.

Examples include:

```
com.google.android.apps.restore  
com.google.android.syncadapters.contacts  
com.google.android.syncadapters.calendar
```

Only remove these packages if you do not use the associated functionality.

4.5.3 Android Components

Optional Android components that can safely be removed.

Examples include:

```
com.android.providers.partnerbookmarks  
com.android.wallpaperbackup  
com.android.wallpapercropper
```

4.5.4 Android Themes

Android theme assets components that can safely be removed, but affect some of the UI.

Examples include:

```
com.android.theme.icon.taperedirect  
com.android.theme.icon.teardrop  
com.android.theme.icon.vessel
```

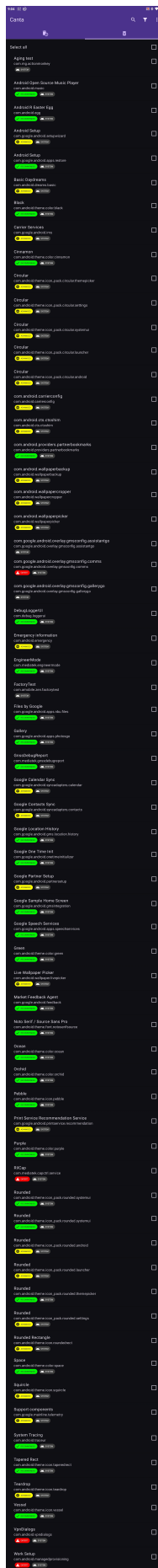


Figure 3: Package selection

Over-
lay
Pack-
ages
Remove
only
the
over-
lay
pack-
ages
rec-
om-
mended.
If you
are
un-
sure
whether
an
over-
lay is
cur-
rently
in
use,
leave
it in-
stalled.

[!TIP] **Uri's Recommendation**

Use the screenshot as the primary source of truth when selecting packages.

It reflect the configuration tested during development of the guide.

4.6 Why Debloat?

Removing unnecessary packages reduces:

- Background CPU usage
- Background memory usage
- Idle battery consumption
- Background services
- System wakeups

A cleaner system leaves more resources available for emulators and frontend applications.

4.7 Reboot the Device

After completing the debloat process:

1. Restart the device.
2. Wait for Android to finish booting.
3. Confirm that the launcher opens normally.
4. Verify that Google Play Store still functions correctly.

4.8 Verification Checklist

Before continuing, verify that:

- ☐ Device booted successfully.
- ☐ Launcher opens normally.
- ☐ Google Play Store works correctly.
- ☐ Essential applications still function.
- ☐ No unexpected crashes occurred.

If everything works as expected, continue with `[[03-System-Settings]]`.

5 03-System-Settings.md

6 Recommended System Settings

Before applying the Advanced ADB optimizations, configure the following Android settings.

These changes reduce unnecessary background activity, improve battery life and complement the optimizations described later in this guide.

Most settings only need to be configured once.

6.1 Google Account

6.1.1 Disable Contact Synchronization

You will probably do not use contacts on your handheld, disable Google Contact Sync. ### Steps

1. Open **Settings**.
2. Navigate to **Accounts**.
3. Select your Google account.
4. Disable **Contacts** synchronization.

This prevents unnecessary background synchronization.

6.2 Google Play

6.2.1 Disable Play Protect

Google Play Protect continuously scans installed applications in the background.

If you only install trusted applications, disabling it reduces unnecessary CPU activity.

6.2.2 Steps

1. Open **Google Play Store**.
2. Tap your profile picture.
3. Open **Play Protect**.
4. Disable **Scan apps with Play Protect**.

[!TIP] Uri's Recommendation

Install all trusted applications before disabling Play Protect.

6.3 Location Services

6.3.1 Disable Location

The device is dedicated to gaming, Location Services are generally unnecessary.

Disable:

- Location
- Google Location Accuracy
- Google Location Sharing

This prevents Android from continuously polling GPS and network-based location services.

6.3.2 Disable Wi-Fi and Bluetooth Scanning

Android can continue scanning for nearby wireless devices even when Wi-Fi or Bluetooth are turned off.

Navigate to:

Settings → Location → Location Services

Disable:

- Wi-Fi Scanning
- Bluetooth Scanning

This reduces unnecessary wireless activity and battery consumption.

6.4 Account Synchronization

6.4.1 Disable Automatic Sync

Automatic synchronization periodically wakes the device to update cloud services.

Navigate to:

Settings → Accounts

Disable:

- Automatically sync app data

This prevents unnecessary background synchronization.

6.5 System Logging

6.5.1 Disable Logging

Diagnostic logging continuously records system events.

If you do not need diagnostic logs, disable the following options:

- Logging Level
- Logger Buffer Size

Both settings can be found using the Settings search function.

Disabling them reduces unnecessary storage activity and background CPU usage.

6.6 Background Applications

6.6.1 Restrict Background Battery Usage

Restrict background execution for applications that do not need to remain active.

For each user-installed application:

1. Open **Settings**.
2. Navigate to **Apps**.
3. Select the application.
4. Open **Battery**.
5. Set **Background Restriction** to **Restricted**.

Repeat this process for every application that does not require background execution.

[!WARNING] Do **not** restrict your launcher or applications that must remain active in the background.

[!TIP] Uri's Recommendation

Uri recommends replacing the stock launcher with a dedicated frontend such as **ES-DE**, **Beacon Launcher** or another launcher designed for handheld gaming.

6.7 Developer Options

6.7.1 Disable Window Animations

Animations improve visual appearance but introduce unnecessary delays when navigating the interface.

Navigate to **Developer Options** and configure:

- Window Animation Scale → **Off**
- Transition Animation Scale → **Off**
- Animator Duration Scale → **Off**

Disabling animations improves the perceived responsiveness of the system.

6.7.2 Keep Hardware Overlays Enabled

Do **not** disable **Hardware Overlays**.

Android's Hardware Composer already manages display composition efficiently.

[!TIP] Uri's Recommendation

Many Android optimization guides recommend disabling Hardware Overlays.

During testing, Uri found no measurable benefit on the PAM. In some cases, disabling them may actually increase GPU workload.

6.8 Before Continuing

Verify that:

- ☐ Play Protect is disabled.
- ☐ Location Services are disabled (if not required).
- ☐ Wi-Fi and Bluetooth scanning are disabled.
- ☐ Automatic synchronization is disabled.
- ☐ Logging is disabled.
- ☐ Background restrictions have been configured.

❑ Window animations are disabled.

The device is now ready for the [[04-01-Advanced-ADB-Toolkit]].

[!WARNING] BELOW THIS POINT THE GUIDE IS STILL PENDING REVISION. FOLLOW WITH CAUTION.

7 04-Advanced-ADB-Toolkit.md

8 Advanced ADB Optimization Toolkit

The Advanced ADB Optimization Toolkit contains the core optimizations described in Uri's guide.

Unlike the previous chapters, these optimizations modify Android system behavior using **Android Debug Bridge (ADB)** commands.

Each optimization is independent unless otherwise noted, allowing you to apply only the changes that fit your workflow.

[!WARNING] These commands modify Android system settings.

Always review each command before executing it.

Commands should be copied exactly as written.

9 Google Play Services

Google Play Services is one of the most active background components in Android.

The following optimizations reduce unnecessary background activity while preserving normal application compatibility.

9.1 Restrict Google Play Services

Restrict background execution and move Google Play Services to the **Restricted** App Standby Bucket.

This significantly reduces background CPU activity while maintaining compatibility with applications that depend on Google Play Services.

9.1.1 Commands

```
adb shell cmd appops set com.google.android.gms RUN_IN_BACKGROUND ignore
adb shell am set-standby-bucket com.google.android.gms restricted
```

[!TIP] **Uri's Recommendation**

This is one of the highest-impact optimizations in the entire guide.

9.2 Restrict Google Play Store

Restrict background execution of Google Play Store while preserving manual application installation and updates.

9.2.1 Commands

```
adb shell cmd appops set com.android.vending RUN_IN_BACKGROUND ignore
adb shell am set-standby-bucket com.android.vending restricted
```

9.3 Restrict Gboard

Prevent Gboard from performing unnecessary background work while preserving normal keyboard functionality.

9.3.1 Commands

```
adb shell cmd appops set com.google.android.inputmethod.latin RUN_IN_BACKGROUND ignore
adb shell am set-standby-bucket com.google.android.inputmethod.latin restricted
```

9.4 Restrict OTA Updater

Restrict background execution of the stock OTA updater.

Manual firmware updates remain available whenever required.

9.4.1 Commands

```
adb shell cmd appops set com.ayaneo.update RUN_IN_BACKGROUND ignore
adb shell am set-standby-bucket com.ayaneo.update restricted
```

9.5 Frontend Protection

Android may mark inactive applications for suspension during extended idle periods.

Protect your frontend so it remains available when returning from emulators.

9.5.1 Daijisho (GitHub)

```
adb shell am set-inactive com.zykoo.daijishou false
```

9.5.2 Daijisho (Google Play)

```
adb shell am set-inactive com.magneticchen.daijishou false
```

If you use another frontend, replace the package name accordingly.

10 Performance Optimizations

The following optimizations improve application responsiveness while reducing unnecessary runtime work performed by Android.

Most of these commands only need to be executed once.

10.1 Force Ahead-of-Time Compilation

Android normally compiles applications during execution using **Just-In-Time (JIT)** compilation.

This command recompiles installed applications using **Ahead-of-Time (AOT)** compilation, reducing runtime CPU usage and improving responsiveness.

10.1.1 Command

```
adb shell cmd package compile -m speed -a
```

[!TIP] Uri's Recommendation

Run this command again after updating emulators or other performance-critical applications.

10.2 Compile a Single Application

Instead of recompiling every installed application, you can recompile only the application that was recently updated.

10.2.1 Command

```
adb shell cmd package compile -m speed -f <package_name>
```

Example:

```
adb shell cmd package compile -m speed -f com.retroarch
```

10.3 Execute Background Package Optimization

Run Android's background package optimization immediately instead of waiting for the scheduled maintenance job.

10.3.1 Command

```
adb shell cmd package bg-dexopt-job
```

10.4 Disable Application Error Reports

Prevent Android from generating unnecessary application error reports.

10.4.1 Command

```
adb shell settings put secure send_action_app_error 0
```

10.5 Disable Security Reports

Prevent Android from generating background security reports.

10.5.1 Command

```
adb shell settings put system send_security_reports 0
```

10.6 Disable Background Sync During Battery Saver

Prevent automatic account synchronization while Battery Saver is enabled.

10.6.1 Command

```
adb shell settings put global master_sync_allow_background_in_battery_saver 0
```

11 System Resource Management

Android continuously schedules maintenance tasks, background jobs and alarms to support installed applications.

The following optimizations reduce unnecessary background activity while preserving normal system functionality.

11.1 Reduce Background Job Scheduling

Limit the number of background jobs Android may execute simultaneously.

This helps reduce unnecessary CPU activity while maintaining normal foreground performance.

11.1.1 Command

```
adb shell settings put global job_scheduler_quota_controller_max_job_count_bg 3
```

11.2 Enable Cached App Freezer

Allow Android to freeze cached applications instead of terminating them.

Freezing inactive applications reduces memory pressure while allowing them to resume more efficiently when needed.

11.2.1 Command

```
adb shell device_config put activity_manager_native_boot use_freezer true
```

11.3 Prioritize System UI

Increase scheduling priority for Android's System UI.

This helps maintain a responsive interface during periods of high CPU utilization.

11.3.1 Command

```
adb shell setprop sys.use_fifo_ui 1
```

11.4 Disable Captive Portal Detection

Android periodically contacts connectivity servers to verify internet access.

If the device is primarily used for gaming, these checks are generally unnecessary.

11.4.1 Command

```
adb shell settings put global captive_portal_mode 2
```

[!TIP] Uri's Recommendation

This optimization is particularly useful for handheld devices that rarely connect to public Wi-Fi networks requiring captive portal authentication.

11.5 Trim Package Cache

Request Android to immediately reclaim unused package cache storage.

This command helps recover storage space after installing, updating or removing applications.

11.5.1 Command

```
adb shell pm trim-caches 999G
```

11.6 Reduce Alarm Wakeups

Adjust Alarm Manager timing parameters to reduce unnecessary device wakeups.

Fewer wakeups generally improve idle battery consumption.

11.6.1 Command

```
adb shell settings put global alarm_manager_constants "allow_while_idle_short_time=10000,allow_while"
```

11.7 Disable Activity Recognition

Prevent Google Play Services from continuously monitoring physical activity.

Most handheld gaming devices do not benefit from this functionality.

11.7.1 Command

```
adb shell cmd appops set com.google.android.gms ACTIVITY_RECOGNITION ignore
```

11.8 Disable Background Usage Statistics

Prevent Google Play Services from collecting application usage statistics.

11.8.1 Command

```
adb shell cmd appops set com.google.android.gms GET_USAGE_STATS ignore
```

11.9 Disable Device Identifier Access

Restrict unnecessary access to hardware identifiers.

11.9.1 Command

```
adb shell cmd appops set com.google.android.gms READ_DEVICE_IDENTIFIERS ignore
```

11.10 Background Resource Management

At this stage, Android should:

- Execute fewer background jobs.
- Wake the device less frequently.
- Freeze cached applications more efficiently.
- Reduce unnecessary CPU scheduling.
- Minimize background maintenance activity.

These optimizations primarily improve idle efficiency while preserving normal application compatibility.

11.11 Verification

Before continuing:

- ☐ All commands executed successfully.
- ☐ No command returned an error.
- ☐ Google Play Store still opens correctly.
- ☐ Google Play Services continues to function normally.

If everything works as expected, continue with **Google Play Services Hardening**.

12 Google Play Services Hardening

Google Play Services provides essential Android functionality, but it also performs a significant amount of background work.

The following optimizations reduce unnecessary background activity while preserving compatibility with applications that depend on Google Play Services.

[!IMPORTANT] These optimizations are intended for a handheld gaming device running Stock OS.

If you rely on Google location history, activity recognition or similar services, review each command before applying it.

12.1 Disable Background Location Access

Prevent Google Play Services from continuously accessing location-related APIs.

If the device is primarily used for gaming, these services are generally unnecessary.

12.1.1 Commands

```
adb shell cmd appops set com.google.android.gms FINE_LOCATION ignore
adb shell cmd appops set com.google.android.gms COARSE_LOCATION ignore
adb shell cmd appops set com.google.android.gms MONITOR_LOCATION ignore
adb shell cmd appops set com.google.android.gms MONITOR_HIGH_POWER_LOCATION ignore
```

12.2 Disable Activity Recognition

Prevent Google Play Services from monitoring physical activity.

12.2.1 Command

```
adb shell cmd appops set com.google.android.gms ACTIVITY_RECOGNITION ignore
```

12.3 Disable Usage Statistics

Prevent Google Play Services from collecting application usage statistics.

12.3.1 Command

```
adb shell cmd appops set com.google.android.gms GET_USAGE_STATS ignore
```

12.4 Disable Device Identifier Access

Restrict unnecessary access to hardware identifiers.

12.4.1 Command

```
adb shell cmd appops set com.google.android.gms READ_DEVICE_IDENTIFIERS ignore
```

12.5 Disable Wi-Fi and Bluetooth Background Scanning

Prevent Google Play Services from continuously scanning nearby wireless devices.

12.5.1 Commands

```
adb shell cmd appops set com.google.android.gms BLUETOOTH_SCAN ignore
adb shell cmd appops set com.google.android.gms NEIGHBORING_WIFI_SCAN ignore
```

12.6 Restrict Wake Locks

Prevent Google Play Services from unnecessarily waking the device.

Reducing wake locks improves idle battery consumption.

12.6.1 Command

```
adb shell cmd appops set com.google.android.gms WAKE_LOCK ignore
```

12.7 Restrict Automatic Startup

Prevent Google Play Services from automatically starting unnecessary background components.

12.7.1 Command

```
adb shell cmd appops set com.google.android.gms AUTO_START ignore
```

12.8 Restrict Overlay Windows

Prevent Google Play Services from creating unnecessary system overlay windows.

12.8.1 Command

```
adb shell cmd appops set com.google.android.gms SYSTEM_ALERT_WINDOW ignore
```

12.9 Disable Google Feedback

Disable Google's feedback application.

12.9.1 Command

```
adb shell pm disable-user --user 0 com.google.android.feedback
```

12.10 Disable Google Turbo

Disable the Google Turbo service.

12.10.1 Command

```
adb shell pm disable-user --user 0 com.google.android.apps.turbo
```

12.11 Enable Sensor Privacy

Apply additional system-wide sensor privacy restrictions.

12.11.1 Commands

```
adb shell settings put global sensor_privacy 1
adb shell settings put global wifi_scan_always_enabled 0
```

[!TIP] Uri's Recommendation

These commands complement the previous optimizations by reducing unnecessary sensor usage and wireless scanning while maintaining normal application compatibility.

13 Expected Result

After completing this section, Google Play Services should:

- Generate fewer device wakeups.
- Perform fewer background scans.
- Poll location services less frequently.
- Reduce idle CPU activity.
- Consume less battery while the device is idle.

Normal Google Play functionality should remain available.

13.1 Verification

Before continuing:

- ☐ Google Play Store still updates applications.
- ☐ Applications depending on Google Play Services continue to function.
- ☐ No unexpected permission dialogs appear.
- ☐ No recurring Google Play Services errors are displayed.

If any application requires functionality disabled by this section, revert only the corresponding optimization.

Continue with **Display & User Experience**.

14 Display & User Experience

The final optimizations focus on improving interface responsiveness while preserving the Stock OS experience.

Unlike previous sections, these changes primarily affect the user interface rather than background system behavior.

14.1 Disable Window Animations

Android animations improve visual appearance but introduce unnecessary delays during navigation.

Navigate to:

Settings → Developer Options

Configure the following options:

- Window Animation Scale → **Animation Off**
- Transition Animation Scale → **Animation Off**
- Animator Duration Scale → **Animation Off**

Disabling these animations makes application launches and system navigation feel significantly more responsive.

14.2 Keep Hardware Overlays Enabled

Some Android optimization guides recommend disabling Hardware Overlays.

For the PAM, this is **not recommended**.

Android's Hardware Composer already manages display composition efficiently.

[!TIP] **Uri's Recommendation**

Leave **Disable HW Overlays** turned **off**.

During testing, disabling Hardware Overlays provided no measurable performance improvement and, in some scenarios, increased GPU workload.

14.3 SurfaceFlinger Optimizations

Some firmware versions expose additional rendering properties through SurfaceFlinger.

Only apply SurfaceFlinger commands that are explicitly supported by your firmware version and documented in the original guide.

Do not attempt to enable undocumented properties.

14.4 Reboot the Device

After completing all Advanced ADB optimizations, restart the device.

Rebooting ensures that all modified services reload using their updated configuration.

15 Final Validation

After the device finishes booting:

Verify that:

- Google Play Store opens normally.
- Google Play Services operates correctly.
- Your frontend launches successfully.
- Installed emulators open normally.
- No unexpected error messages appear.

Launch one or more demanding games to confirm that:

- Performance is stable.
 - No graphical artifacts are present.
 - Audio behaves normally.
 - Input latency feels consistent.
 - Battery consumption during idle has improved.
-

16 Troubleshooting

16.1 Google Play Store Stops Updating

Verify that:

- Google Play Services remains installed.
- Google Play Store has not been disabled.
- Network connectivity is available.
- Automatic date and time are configured correctly.

If necessary, temporarily revert the related Google Play Services optimization and test again.

16.2 Unexpected Application Behavior

If an application behaves unexpectedly after applying an optimization:

1. Identify the optimization related to that application.
2. Revert only that specific change.
3. Test the application again.

Avoid reverting unrelated optimizations.

16.3 Increased Battery Consumption

Major firmware updates may restore default Android behavior.

Review:

- Recommended System Settings
- Google Play Services optimizations
- Background restrictions

Reapply any settings that were reset by the update.

17 Maintenance

Most optimizations only need to be applied once.

However, consider the following maintenance tasks:

17.1 After Updating an Emulator

Recompile the updated application.

```
adb shell cmd package compile -m speed -f <package_name>
```

17.2 After Major Firmware Updates

Review:

- Developer Options
- Background Restrictions
- Google Play Services functionality

Some settings may need to be reconfigured.

17.3 Periodic Review

Occasionally verify that:

- Background restrictions are still applied.
 - Newly installed applications are configured appropriately.
 - Google Play Services continues functioning as expected.
-

18 Completion Checklist

Before considering the optimization process complete:

- ☐ Device successfully rebooted.
- ☐ Google Play Store functioning normally.
- ☐ Google Play Services functioning normally.
- ☐ Frontend launching correctly.
- ☐ Emulators functioning correctly.
- ☐ Performance validated.
- ☐ Battery behavior validated.

Congratulations.

You have completed the **Advanced ADB Optimization Toolkit**.

Continue with **05 – Game Driver Configuration**.

19 05-Game-Driver.md

20 Game Driver Configuration

After completing the system optimizations, configure the appropriate graphics driver for your emulators.

The Stock OS allows individual applications to use alternative graphics drivers, which may improve rendering performance and compatibility depending on the emulator and graphics API being used.

[!IMPORTANT] Graphics driver behavior may vary depending on the firmware version, emulator version and graphics backend.

Always test each emulator after changing its driver.

20.1 Open Game Driver Preferences

Navigate to:

Settings → System → Developer Options → Game Driver Preferences

The list displays applications that support selecting an alternative graphics driver.

20.2 Configure Each Emulator

Select each supported emulator individually.

Apply the graphics driver recommended in the original guide.

If multiple drivers are available, use the one specified for your device and firmware version.

Repeat the process for every emulator you actively use.

20.3 Validate the Configuration

After changing the driver:

1. Launch the emulator.
2. Load a demanding game.
3. Allow gameplay for several minutes.
4. Observe overall stability.

Verify that:

- Graphics render correctly.
 - Audio behaves normally.
 - No graphical artifacts appear.
 - Performance remains stable.
 - No unexpected crashes occur.
-

[!TIP] Uri's Recommendation

If changing the graphics driver introduces graphical issues or instability, revert the emulator to the default system driver.

20.4 Compatibility Notes

Different emulators may respond differently depending on:

- Android version
- Stock OS version
- GPU firmware
- Graphics driver version
- Graphics backend (Vulkan or OpenGL)

For this reason, always validate changes using the games you play most frequently.

20.5 Final System Validation

At this point, verify that:

- The launcher starts normally.
 - Google Play Store functions correctly.
 - Google Play Services operates normally.
 - All configured emulators launch successfully.
 - No unexpected warnings or error messages are displayed.
-

21 Optimization Complete

Congratulations.

You have completed the optimization process described in this guide.

Your device should now benefit from:

- Reduced background activity
- Lower idle battery consumption
- Improved system responsiveness
- Better resource availability for emulation
- A cleaner and more consistent Stock OS environment

Individual results may vary depending on firmware version, installed applications and emulator configuration.

22 Ongoing Maintenance

Most optimizations only need to be performed once.

However, after major firmware updates, factory resets or significant application changes, review the following:

- Background restrictions
- Developer Options
- Game Driver configuration
- AOT compilation
- Google Play functionality

Reapply any settings that were reset.

22.1 Completion Checklist

Before considering the process complete, confirm that:

- ☐ Device rebooted successfully.
- ☐ System boots without errors.
- ☐ Google Play Store works correctly.
- ☐ Google Play Services functions normally.
- ☐ Frontend launches correctly.
- ☐ All emulators tested successfully.
- ☐ Performance validated.
- ☐ Battery behavior validated.

You have successfully completed the **PAM Stock OS Optimization Guide**.

Continue to the **Appendices** for reference material, troubleshooting information and package references.

23 06-Appendices.md

24 Appendices

This section contains reference material that complements the optimization guide.

The information presented here is intended for quick consultation during installation, maintenance and troubleshooting.

25 Appendix A — Common Package Names

The following package names are referenced throughout this guide.

Application	Package Name	Notes
Google Play Services	<code>com.google.android.gms</code>	Core Google services
Google Play Store	<code>com.android.vending</code>	Google Play application
Gboard	<code>com.google.android.inputmethod.latin</code>	Google Keyboard
Daijisho (Google Play)	<code>com.magneticchen.daijishou</code>	Google Play version
Daijisho (GitHub)	<code>com.zykoo.daijishou</code>	GitHub release
RetroArch	<code>com.retroarch</code>	Example used for AOT compilation
ES-DE	<i>Verify installed package name</i>	Package name may vary
Beacon Launcher	<i>Verify installed package name</i>	Package name may vary

26 Appendix B — Common ADB Commands

The commands below are the most frequently used throughout this guide.

26.1 Compile All Installed Applications

```
adb shell cmd package compile -m speed -a
```

26.2 Compile a Single Application

```
adb shell cmd package compile -m speed -f <package_name>
```

Example:

```
adb shell cmd package compile -m speed -f com.retroarch
```

26.3 Execute Background Package Optimization

```
adb shell cmd package bg-dexopt-job
```

26.4 Trim Package Cache

```
adb shell pm trim-caches 999G
```

26.5 Reboot the Device

```
adb reboot
```

27 Appendix C — Glossary

27.1 ADB

Android Debug Bridge

A command-line interface used to communicate with Android devices.

27.2 AOT

Ahead-of-Time Compilation

A compilation method that prepares applications before execution, reducing runtime CPU usage.

27.3 JIT

Just-in-Time Compilation

Applications are compiled while they are running instead of beforehand.

27.4 App Standby Bucket

Android mechanism that determines how aggressively an application may execute background work.

27.5 AppOps

Android permission management framework used to control specific application operations.

27.6 Background Restriction

Android feature that limits background execution for selected applications.

27.7 Canta

Application used together with Shizuku to remove or disable system packages without root access.

27.8 Developer Options

Advanced Android configuration menu intended for developers and power users.

27.9 Shizuku

Application that allows supported apps to access privileged Android APIs without requiring root access.

27.10 SurfaceFlinger

Android system compositor responsible for rendering and composing the display.

27.11 Hardware Composer

Android component responsible for efficiently composing graphical layers before displaying them.

27.12 Wake Lock

Android mechanism that allows an application to keep the CPU awake while performing background work.

28 Troubleshooting

28.1 Google Play Store Does Not Update Applications

Verify that:

- Google Play Services remains enabled.
- Google Play Store has not been disabled.
- Network connectivity is available.
- Automatic date and time are configured correctly.

If necessary, temporarily revert the corresponding optimization and test again.

28.2 Google Play Services Displays Errors

Review the **Google Play Services Hardening** section.

If required, revert the most recently applied optimization and verify whether the issue is resolved.

28.3 Emulator Performance Did Not Improve

Performance improvements depend on multiple factors, including:

- Emulator version
- Graphics driver
- Graphics backend (Vulkan or OpenGL)
- Firmware version
- Game compatibility

Not every emulator or game will benefit equally from every optimization.

28.4 System Behaves Unexpectedly

If an issue appears after applying an optimization:

1. Identify the most recent change.
2. Revert only that specific optimization.
3. Test the device again.

Avoid reverting unrelated optimizations.

28.5 Firmware Update Reset Some Settings

Major firmware updates may restore default Android configuration.

Review the following chapters:

- Recommended System Settings
- Advanced ADB Optimization Toolkit
- Game Driver Configuration

Reapply any settings that were reset.

29 Credits

29.1 Original Guide

Uri

Original research, testing methodology, optimization strategy and technical documentation.

29.2 Community Edition

BruhMeh

Editorial review, Markdown conversion, documentation formatting, OCR corrections, repository organization and publishing support.

30 Final Notes

The purpose of this Community Edition is to preserve the original optimization guide while making it easier to read, review and maintain.

Technical recommendations remain those of the original author unless explicitly documented.

Community contributions should prioritize:

- Accuracy
- Readability
- Maintainability
- Respect for the original work

Thank you for helping preserve and improve this documentation.